

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 03 -

DESENVOLVIMENTO E ENGENHARIA DE MODELOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	27
O QUE VOCÊ VIU NESTA AULA?	30
REFERÊNCIAS.....	32

EMENDAS

O QUE VEM POR AÍ?

Imagine que você já construiu um modelo inicial (um MVP de modelo) que alcançou cerca de 80% de acurácia na detecção de doença cardíaca – um bom começo, mas ainda longe do ideal. E se pudéssemos melhorar esse desempenho? Como extrair outros sinais dos dados, escolher algoritmos mais eficazes e ajustar as configurações do modelo para torná-lo mais preciso e robusto? Nesta aula, vamos explorar exatamente esses próximos passos do ciclo de vida de um modelo.

Veremos como refinar os atributos (features) disponíveis – criando novas variáveis a partir de dados brutos e selecionando as mais relevantes – e como isso pode turbinar os resultados preditivos. Também vamos comparar diferentes algoritmos de aprendizado (de modelos lineares a árvores de decisão e ensembles) para entender seus prós e contras e descobrir qual se adapta melhor ao nosso problema. Tudo isso nos levará a construir experimentos bem estruturados, que nos auxiliam na construção de modelos mais robustos, prevenindo erros e preparando o ambiente para a implementação em aplicações práticas.

Ao longo desta aula, usando novamente o conjunto de dados de Doença Cardíaca da UCI (Heart Disease) – o mesmo que serviu de base para nosso MVP – vamos aplicar metodologias avançadas de desenvolvimento de modelos. Você aprenderá a explorar técnicas de transformação de atributos de entrada dos modelos (feature engineering) usando conhecimentos do domínio de saúde, a reduzir a dimensionalidade do problema via seleção de atributos redundantes e a experimentar algoritmos mais sofisticados (como Random Forest) para tentarmos superar o baseline estabelecido pelo modelo mais simples.

Além disso, vamos ajustar hiperparâmetros para extrair o máximo desempenho possível de um modelo e estruturar nosso código de maneira modular, facilitando a manutenção e a eventual implantação em sistemas reais. Ao final da aula, você terá uma visão sólida de como evoluir de um modelo básico para uma solução de aprendizado de máquina bem calibrada, documentada e pronta para os desafios de produção. Preparado(a) para levar seu modelo ao próximo nível? Então vamos em frente!

HANDS ON

Agora é hora de colocar a mão na massa e evoluir nosso projeto de detecção de doenças cardíacas. Anteriormente, construímos um MVP de modelo (Regressão Logística) que serviu como baseline inicial (~80% de acurácia). Nosso objetivo agora é superar esse baseline aprimorando o conjunto de features e testando modelos mais complexos. Nesta atividade prática, seguiremos uma série de passos: desde criar e selecionar atributos, passando por comparar algoritmos de classificação diferentes, até ajustar hiperparâmetros e montar um pipeline final. Usaremos novamente Python com bibliotecas populares (Pandas, Scikit-learn etc.) para implementar as técnicas. **Na seção Saiba Mais, detalharemos os passos principais.**

Certifique-se de ter o ambiente configurado com as mesmas bibliotecas usadas anteriormente (Pandas, NumPy, Scikit-learn etc.). Vamos reutilizar o dataset pré-processado da aula anterior (com variáveis codificadas e dados limpos). Separe novamente em variáveis preditoras X e alvo y e então em conjuntos de treino e teste (por exemplo, 80% treino, 20% teste) e utilize um `random_seed` fixo no `train_test_split`. Assim, poderemos comparar os resultados dos novos modelos com o baseline de forma justa.

SAIBA MAIS

Passo 1: Feature Engineering e Seleção de Atributos

Vamos criar novas features e avaliar a importância das existentes, explorando transformações não lineares e combinações de variáveis. Por exemplo, podemos dar maior peso à idade usando uma transformação quadrática para capturar um risco que aumenta de forma acelerada. Também pode ser útil criar features como a relação entre Colesterol e Idade, que pode indicar um risco relativo, ou a razão entre a Pressão Arterial de Repouso e o Colesterol, que pode sinalizar um perfil de risco vascular combinado.

Além disso, podemos agrupar a idade em faixas (bins) para capturar padrões não lineares de idade vs. risco ou criar uma variável categorizando o nível de depressão do segmento ST (oldpeak) em "baixo", "moderado" ou "alto". Também é possível combinar informações: no domínio médico, uma ideia seria cruzar o tipo de dor no peito com a ocorrência de angina induzida por exercício, formando uma feature composta (por exemplo, um código que indica pacientes com dor assintomática e sem angina induzida – possivelmente um subgrupo de menor risco).

```
# Dar maior peso a idade (quadrática)
df_engineered['age_squared'] = df_engineered['age'] ** 2

# Relação colesterol/idade: pode indicar risco relativo
df_engineered['cholesterol_to_age'] =
df_engineered['chol'] / (df_engineered['age'] + eps) # Evitar
divisão por zero

# Percentual da frequência cardíaca máxima alcançada (regra
220 - age)
# Interpretação: quanto próximo do máximo previsto o paciente
chegou no esforço
if 'thalch' in df_engineered.columns and 'age' in
df_engineered.columns:
    predicted_max_hr = (220 -
df_engineered['age']).clip(lower=1) # evita divisão por zero
    df_engineered['max_hr_pct'] = df_engineered['thalch'] /
(predicted_max_hr + eps)

# Razão pressão/colesterol: pode sinalizar perfil de risco
vascular relativo
```

```
if 'trestbps' in df_engineered.columns and 'chol' in
df_engineered.columns:
    df_engineered['bp_chol_ratio'] =
df_engineered['trestbps'] / (df_engineered['chol'] + 1)
```

Código-fonte 1 - Exemplo de Feature Engineering (Python)

Fonte: Elaborado pelo autor (2026)

Após criar novas colunas no DataFrame no processo de feature engineering, podemos padronizar e normalizar os dados conforme necessário. Por exemplo, modelos lineares e métodos baseados em distância (k-NN, SVM) se beneficiam de features numéricas escaladas – podemos aplicar StandardScaler do Scikit-learn nos atributos contínuos (pressão, colesterol etc.) para que fiquem na mesma escala. Em seguida, realizaremos uma seleção de atributos para reduzir a dimensionalidade e eliminar variáveis pouco informativas.

Podemos empregar um método de filtro univariado, como o qui-quadrado ou ANOVA, utilizando o SelectKBest do Scikit-learn para selecionar, digamos, as X melhores features correlacionadas com o alvo. Alternativamente, podemos inspecionar a importância de features usando um modelo de árvore de decisão inicial, como por exemplo treinar um RandomForestClassifier rápido nos dados de treino extraíndo a importância das features, listando as variáveis em ordem de importância e considerando descartar aquelas com importância muito baixa ou que façam menos sentido para especialistas. Lembre-se: qualquer procedimento de seleção deve ser feito somente com dados de treino, evitando olhar o conjunto de teste para não viciar a avaliação.

```
# Seleção de features usando ANOVA F-value
from sklearn.feature_selection import SelectKBest,
f_classif

selector_preview = SelectKBest(f_classif, k=min(25,
X_train.shape[1]))
X_train_selected = selector_preview.fit_transform(X_train,
y_train)
```

Código-fonte 2 - Exemplo de Seleção de Features utilizando o SelectKBest (máx: 25 variáveis)
(Python)

Fonte: Elaborado pelo autor (2026)

Com as features selecionadas, gere um novo conjunto X_train_sel e X_test_sel contendo apenas essas colunas.

Passo 2: Treinamento de diferentes modelos

Agora, vamos treinar alguns modelos diferentes e comparar seu desempenho. Além da Regressão Logística (já conhecida), treinaremos, por exemplo, uma Árvore de Decisão e um Random Forest nos dados selecionados. Para cada um, precisamos ajustar os dados de treino e computar as métricas no conjunto de teste (acurácia, f1_score etc.). É importante usar a mesma partição de treino/teste e o mesmo random_state para todos, garantindo que possamos compará-los de forma justa. Caso prefira, utilize validação cruzada no conjunto de treino para estimar a performance média de cada algoritmo de forma mais robusta.

```
# Exemplo ajuste de modelo Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train_selected, y_train)

# Exemplo ajuste de modelo Random Forest
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train_selected, y_train)

# Exemplo ajuste de modelo SVM
svm = SVC(probability=True, random_state=42)
svm.fit(X_train_selected, y_train)
```

Código-fonte 3 - Exemplos de ajustes de diferentes modelos (Python)
Fonte: Elaborado pelo autor (2026)

Ao comparar os resultados, verifique qual modelo obteve a maior métrica escolhida e se alguma apresenta vantagem significativa em precisão ou recall, dependendo do que for mais crítico (em saúde, por exemplo, um bom recall pode ser vital para não perder casos positivos). Você provavelmente observará diferenças: modelos de árvores e random forest tendem a capturar relações não lineares entre as features e podem superar o modelo linear simples. No nosso cenário, é esperado que o Random Forest apresente uma acurácia superior ao baseline – digamos, hipoteticamente, algo em torno de 85% no teste, superando os ~80% da regressão logística. Essa melhoria indicaria que as features adicionais ou transformadas estão trazendo valor e que o algoritmo mais complexo está conseguindo aproveitar padrões nos dados que o modelo linear não capturava.

Vale notar também aspectos como tempo de treinamento e interpretabilidade: uma árvore de decisão simples é facilmente interpretável (podemos visualizá-la),

enquanto um Random Forest é mais complexo, porém geralmente mais preciso. Comparar algoritmos nos lembra que não existe um modelo único que seja o “melhor” para todos os problemas – o desempenho depende dos dados e do contexto. Por isso, testar abordagens distintas é uma prática fundamental no desenvolvimento de modelos.

Passo 3: Tuning de Hiperparâmetros

Escolhido um modelo promissor (por exemplo, o Random Forest, caso tenha se saído melhor), partimos para ajustá-lo finamente. Os hiperparâmetros são configurações do algoritmo que controlam sua complexidade e modo de aprendizagem (por exemplo, número de árvores e profundidade máxima em um Random Forest, valor de K no k-NN, C e kernel em uma SVM etc.). Em vez de confiar nos valores default da biblioteca, vamos realizar uma busca em grade (grid search) para encontrar a combinação ótima de hiperparâmetros para nosso conjunto de dados.

Utilizando a classe GridSearchCV do Scikit-learn, definimos um dicionário de parâmetros a testar – por exemplo, { 'n_estimators': [50, 100, 200], 'max_depth': [None, 4, 8] } para o Random Forest – e executamos a busca com validação cruzada no conjunto de treino. O Scikit-learn avaliará exaustivamente todas as combinações (neste exemplo, $3 \times 3 = 9$ configurações) usando, por padrão, 5 folds de CV, totalizando 45 treinos de modelo. Ao final, podemos inspecionar grid.best_params_ para analisar a melhor configuração encontrada e então treinar um modelo final com esses hiperparâmetros ótimos.

```
# Tuning básico de Random Forest
rf_pipeline = Pipeline([
    ("model", RandomForestClassifier(random_state=42))
])
param_grid = {
    "model__n_estimators": [50, 100, 200],
    "model__max_depth": [None, 4, 8]
}
grid_search = GridSearchCV(
    estimator=rf_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
```



```
n_jobs=-1
)
grid_search.fit(X_train, y_train)
best_simple_rf = grid_search.best_estimator_
print("Melhores parâmetros:", grid_search.best_params_)
```

Código-fonte 4 - Exemplo de Otimização de Hiperparâmetros de um modelo RandomForest (Python)
Fonte: Elaborado pelo autor (2026)

Alternativamente, poderíamos usar uma busca aleatória (RandomizedSearchCV), utilizando amostras de combinações aleatórias de uma distribuição de valores – abordagem útil quando o espaço de parâmetros é muito grande, pois pode encontrar bons resultados com menos tentativas. Para nossa tarefa, suponha que a busca identifique, por exemplo, que `max_depth=4` e `n_estimators=100` otimizam a performance do Random Forest. Com o modelo ajustado, avalie novamente o conjunto de testes. É esperado uma melhoria modesta em relação aos parâmetros default – por exemplo, a acurácia pode subir alguns pontos percentuais adicionais e métricas como F1 podem se aproximar ainda mais do ideal.

Aqui também devemos ter cuidado para não ter o overfitting nos dados de treino durante o tuning; por isso usamos validação cruzada. Overfitting é quando o modelo memoriza o conjunto de dados de treino, incluindo o ruído, em vez de aprender o padrão geral. O modelo fica excessivamente ajustado aos detalhes específicos da amostra de treino. Uma boa prática avançada é empregar cross-validation para validar o processo de tuning em si, mas isso pode ser custoso computacionalmente. De todo modo, o tuning de hiperparâmetros é um passo essencial para extrair o máximo de um algoritmo – muitas vezes a diferença entre um modelo mediano e um ótimo está em escolher bem parâmetros como regularização, profundidade ou taxa de aprendizagem (no caso de modelos de boosting).

Passo 4: Construção de um Pipeline Reproduzível

Conforme nossas etapas vão se acumulando (pré-processamento, seleção de features, treinamento, predição), torna-se importante organizar tudo isso em um pipeline único, que garanta que as transformações apropriadas sejam aplicadas tanto nos dados de treino quanto nos de teste/produção de forma consistente. Vamos utilizar a classe Pipeline do Scikit-learn para encadeamento das etapas.

Por exemplo, podemos montar um pipeline que primeiro aplique o StandardScaler e/ou o SelectKBest (ajustado com os dados de treino) e em seguida faça o fit do modelo final otimizado. Assim, ao chamar `pipeline.fit(X_train, y_train)`, todas as outras etapas dependentes são executadas na ordem e o mesmo ocorre na predição: `pipeline.predict(X_test)` automaticamente realizará transform nos dados de teste usando os parâmetros calculados no treino (por exemplo, normalizando e escalonando a média e desvio do treino, selecionando as mesmas melhores features encontradas no treino e assim por diante).

Isso evita erros comuns, como aplicar acidentalmente uma normalização dos dados de treino e esquecer de fazer o mesmo no teste – um problema que pode distorcer completamente a avaliação do modelo. Além disso, pipelines facilitam o uso de validação cruzada e grid search em múltiplas etapas ao mesmo tempo, pois tratam todo o conjunto de passos como um estimador único. No nosso caso, podemos inclusive integrar o tuning de hiperparâmetro no pipeline (usando Pipeline + GridSearchCV conjuntamente) para buscar, por exemplo, o melhor número de features no SelectKBest e os melhores parâmetros do modelo final, tudo em uma única passagem de CV.

Após montar o pipeline, testamos uma previsão final no holdout (conjunto de teste separado) e obtemos as métricas definitivas do nosso modelo desenvolvido. Anote esses resultados – digamos que agora tenhamos alcançado 88% de acurácia e 0.88 de F1 no teste (valores ilustrativos), uma melhoria substancial em relação ao MVP inicial. O pipeline construído é reutilizável: podemos salvar esse objeto (usando a biblioteca Python pickle ou joblib) para carregá-lo depois e gerar previsões em novos dados, garantindo que todas as etapas de processamento sejam aplicadas corretamente.

Passo 5: Boas Práticas de Código e Integração

Por fim, ao desenvolver um modelo de forma profissional, é crucial organizar o código de modo que ele seja compreensível, versionado e fácil de integrar em aplicações. Durante o hands on, evitamos escrever tudo em um único notebook gigantesco. Em vez disso, poderíamos estruturar o projeto conforme padrões recomendados – por exemplo, seguindo um template de projeto de ciência de dados

(como o fornecido pelo Cookiecutter Data Science), onde há pastas separadas para dados brutos, dados processados, notebooks exploratórios e código modular.

Poderíamos criar um módulo Python, por exemplo, `preprocessing.py` com funções para tratar os dados (normalização, encoding etc.), outro `features.py` contendo a lógica de criação/seleção de features e um `train.py` para treinar o modelo dado um conjunto de hiperparâmetros. Essa modularização torna o código mais limpo e reutilizável – se amanhã quisermos testar um modelo em um novo dataset de saúde, podemos reaproveitar boa parte dessas funções sem reescrever tudo.

Durante o hands on, também prestamos atenção na reprodutibilidade: fixamos sementes aleatórias (como o `random_state=42`) em divisões de dados e modelos e registramos os resultados de cada experimento. Você talvez tenha notado o uso do MLflow no código do MVP da aula anterior e nos exemplos desta aula que serão disponibilizados posteriormente – trata-se de uma ferramenta de rastreamento de experimentos que nos permitiu logar métricas de treino e teste automaticamente. Mantivemos essa prática no desenvolvimento avançado: cada execução (run) de treino com um conjunto de features ou hiperparâmetros foi registrada, possibilitando comparações objetivas entre experimentos. Ao final, dispomos de um histórico completo de como chegamos ao modelo final, o que é valioso para auditoria e aprendizado. O MLFlow será explorado em maiores detalhes na aula sobre o processo de implantação de modelos (deployment).

Concluindo, temos agora um modelo muito mais elaborado do que no início: utilizamos engenharia de atributos para extrair mais sinal dos dados, filtramos variáveis irrelevantes para simplificar o modelo, escolhemos um algoritmo mais potente com base em evidências experimentais e ajustamos seus parâmetros para maximizar o desempenho. Tudo isso foi feito seguindo boas práticas de desenvolvimento, produzindo um pipeline reproduzível e um código bem organizado. No próximo segmento, aprofundaremos os conceitos teóricos por trás dessas etapas e discutiremos por que essas técnicas funcionam e quando utilizá-las.

Após praticar o desenvolvimento avançado do modelo, é hora de consolidar e expandir os conceitos teóricos envolvidos. Vamos entender com mais profundidade por que as técnicas de engenharia e seleção de features melhoram os modelos, em que situações um algoritmo supera outro, como abordar o ajuste de hiperparâmetros de forma eficiente e a importância de estruturar bem o processo de modelagem. Além

disso, discutiremos o contexto maior: como as escolhas de modelagem estão alinhadas ao problema de negócio e às características dos dados disponíveis. Tudo isso dará embasamento para você tomar decisões informadas em projetos reais de Machine Learning.

Entendendo o Problema de Negócio e os Dados

Desenvolver um modelo de Machine Learning não é apenas um exercício técnico – compreender o objetivo do negócio e a natureza dos dados é fundamental em todas as etapas. Antes mesmo de adicionar novas features ou testar algoritmos, devemos questionar: quais sinais nos dados realmente importam para o resultado que queremos prever? No caso da saúde cardíaca, por exemplo, saber que a variável chol (colesterol) está elevada em pacientes com doença nos diz algo, mas talvez a combinação de colesterol alto com idade avançada seja um indicador muito mais forte de risco. Essa intuição vem do conhecimento de domínio (médico, neste caso).

Portanto, conversar com especialistas e mergulhar na literatura do problema pode sugerir features derivadas úteis (como índices de risco) ou alertar sobre quais variáveis são irrelevantes. Lembre-se: por mais poderoso que seja um algoritmo, ele só aprenderá o que for fornecido como input. Garbage in, garbage out! Uma engenharia de atributos eficaz deve capturar os aspectos do problema que fazem sentido no mundo real. Do ponto de vista de negócio, também é crucial alinhar a métrica de sucesso do modelo com os objetivos reais.

No nosso exemplo, se a meta de negócio é identificar o máximo possível de doentes para encaminhá-los a exames complementares, então otimizar o recall pode ser mais importante que otimizar a acurácia. Essas considerações orientam escolhas como penalizar diferentemente erros de um tipo (falso negativo) em vez de outro. Além disso, conhecer os limites e fontes dos dados (proveniência, possíveis vieses, atualizações futuras) ajuda a planejar um modelo mais robusto. Em suma, o desenvolvimento e a engenharia de modelos eficaz passam por um ciclo de entendimento do problema -> experimentação técnica -> retorno ao problema: constantemente verificamos se o que o modelo está aprendendo faz sentido e atende ao uso esperado.

Seleção de Features

Seleção de features é o processo de identificar quais atributos do conjunto de dados são mais relevantes para a tarefa preditiva e eliminar aqueles desnecessários ou redundantes. Por que isso é importante? Primeiro, por uma questão de generalização: usar muitas variáveis irrelevantes pode fazer o modelo aprender “ruídos” específicos do dataset de treino, prejudicando seu desempenho em novos dados. Ao remover features supérfluas, reduzimos a dimensionalidade e, portanto, o risco de overfitting.

Segundo, há o ganho em eficiência – menos variáveis significa modelos mais leves e rápidos, algo crucial quando pensamos em tempo de predição, memória ou até consumo de energia (em aplicações embarcadas, por exemplo). E terceiro, a seleção ajuda na interpretabilidade: modelos com menos variáveis são mais fáceis de explicar para stakeholders (ex.: “as cinco características mais importantes para prever risco cardíaco são X, Y, Z...”). Existem diversas técnicas para seleção de features, geralmente classificadas em três tipos: métodos de filtro, wrapper e embedded.

Nos métodos filtro, aplicamos análises estatísticas simples antes da etapa de modelagem. Por exemplo, podemos calcular a variância de cada feature e eliminar aquelas com variância muito baixa – isto é, que quase não mudam de amostra para amostra. A intuição é que um atributo constante ou quase constante não contribui para distinguir casos. Outra estratégia filtro é medir a correlação de cada feature com a variável alvo (no caso de classificação binária, podemos usar testes como qui-quadrado ou ANOVA F-value para cada atributo individual) e rankeá-las por essa correlação.

O SelectKBest que utilizamos no hands on é exatamente isso: ele seleciona os K melhores atributos segundo um teste univariado pré-definido. Esses métodos são rápidos e independentes de qualquer algoritmo de aprendizagem específico, mas consideram cada feature isoladamente, podendo ignorar combinações que, embora individualmente fracas, juntas são úteis.

Nos métodos wrapper, a seleção de features ocorre através de uma busca guiada por um modelo de aprendizado. O exemplo clássico é o RFE (Recursive Feature Elimination): ele treina um modelo (por exemplo, uma regressão linear ou

árvore) usando todas as features e avalia seu desempenho; então remove a feature menos importante segundo os coeficientes/pesos do modelo; retreina sem essa feature e repete o processo iterativamente até restar um número desejado de features. O RFE explora subsets de variáveis e usa o modelo como parte do critério, tendendo a encontrar conjuntos mais adequados para aquele algoritmo em particular. O custo computacional é mais alto (pois envolve treinar múltiplos modelos), mas frequentemente resulta em melhores subsets que os métodos filtro, pois considera interações entre variáveis no desempenho do modelo.

Nos métodos embedded, a seleção de features acontece durante o próprio treinamento do modelo. Alguns algoritmos têm mecanismos internos de escolha de atributos: por exemplo, árvores de decisão naturalmente fazem uma seleção ao dividir nós pelas features mais informativas; um Random Forest fornece importâncias de features médias que servem para rankear variáveis; modelos lineares com regularização L1 (Lasso) tendem a zerar os coeficientes de variáveis menos relevantes, efetivamente realizando seleção. O Scikit-learn disponibiliza métodos como o `SelectFromModel` para aproveitar esses modelos treinados – por exemplo, usar um `RandomForest` ajustado para selecionar features com importância acima de um certo limite. Essa abordagem embedded é eficiente, pois integra o processo, mas depende de escolher bem o modelo base e seus hiperparâmetros (por exemplo, uma regularização muito forte L1 pode eliminar variáveis demais).

Independentemente do método, um ponto crucial é evitar o leakage de dados durante a seleção. Conforme mencionado, sempre dividimos os dados de treino e teste antes de fazer seleção de features. Incluir informações do teste na escolha de variáveis pode levar a um viés otimista: você acaba selecionando atributos “especiais” que por acaso correlacionam-se com o alvo no dataset inteiro, mas que não generalizam (um exemplo extremo: selecionar atributos randomicamente correlacionados ao target em todo o conjunto – isso inclui influência do teste – pode aumentar artificialmente a performance).

Seguindo as práticas adequadas (feature selection ajustada apenas em cima do treino e aplicada no teste via pipeline), garantimos uma estimativa fiel do poder preditivo real. Em resumo, a seleção de features deve buscar o equilíbrio entre simplicidade e preditividade: queremos o menor conjunto de variáveis que ainda detenha a maior parte da informação relevante do problema.

Engenharia de Atributos (Feature Engineering)

Se a seleção de features trata de reduzir variáveis, a engenharia de atributos trata de criar ou transformar variáveis para melhorar a representação do problema para o modelo. É aqui que a criatividade e o conhecimento de domínio entram em cena. Feature Engineering é o processo de selecionar, modificar e criar características a partir de dados brutos para melhorar o desempenho de modelos de aprendizado. Enquanto algoritmos modernos (como redes neurais profundas) prometem extrair automaticamente representações relevantes dos dados, em muitos cenários de aprendizado supervisionado tradicional a intervenção humana na construção de features ainda faz uma diferença enorme.

Algumas técnicas comuns de engenharia de atributos incluem:

- **Transformações matemáticas:** aplicar transformações em variáveis numéricas para ajustar suas distribuições. Exemplos: usar escala logarítmica para atenuar a influência de outliers (ex.: $\text{oldpeak_log} = \log(1 + \text{oldpeak})$) para diminuir o peso de valores muito altos de depressão ST) ou extrair partes de uma variável (ex.: de uma data completa, extrair apenas o ano ou o mês, se fizer sentido). Transformações também envolvem escalonamento e normalização – padronizar features para média 0 e desvio 1 (StandardScaler) ou escalar para um intervalo [0,1] (MinMaxScaler). Isso é particularmente útil porque alguns algoritmos supõem que as features estejam aproximadamente na mesma escala (ex.: regressão logística, SVM, redes neurais) para convergirem melhor. No dataset de doença cardíaca, por exemplo, podemos normalizar os níveis de colesterol e pressão arterial para evitar que suas magnitudes diferentes influenciem indevidamente o cálculo de distâncias em um k-NN.
- **Criação de features combinadas:** formar novas features a partir de combinações de duas ou mais originais. Poderia ser uma soma, produto, razão ou qualquer relação que faça sentido. No contexto cardíaco: calcular a “taxa colesterol/idade” – um paciente jovem com colesterol de 300 pode ser mais preocupante que um idoso com colesterol de 300, então essa razão poderia capturar parte do risco relativo. Outro exemplo: multiplicar thalach (frequência cardíaca máxima) por age – talvez frequências altas em idosos

sejam mais indicativas de estresse cardíaco do que em jovens. São hipóteses que podemos testar. Ferramentas automatizadas como o PolynomialFeatures do Scikit-learn geram todas combinações polinomiais até certo limite (por exemplo, todas multiplicações de 2 a 3 variáveis), mas isso pode aumentar muito a dimensionalidade, então é preciso critério. Muitas vezes, menos é mais: features criadas manualmente com base em conhecimento de causa podem superar dezenas de features polinomiais cegamente geradas.

- **Encoding de variáveis categóricas:** converter informações categóricas em forma numérica útil para modelos. Já fizemos isso no MVP, usando one-hot encoding via `pd.get_dummies` para variáveis como tipo de dor no peito, resultados do exame thal etc. Lembre-se que one-hot encoding cria colunas binárias para cada categoria, permitindo que modelos numéricos as usem. Porém, há outros encodings possíveis: ordinais (quando as categorias têm ordem, como “baixa, média, alta”), target encoding (substituir categoria pela taxa de evento, útil mas requer cuidado para não viciar) e técnicas de embedding mais avançadas. No caso do nosso dataset, as categorias principais já foram transformadas, mas poderíamos melhorar: note que thal originalmente tinha valores 3,6,7 indicando normal, defeito fixo, defeito reversível. Em vez de deixar o modelo “descobrir” a ordem, poderíamos criar manualmente uma variável binária “thal_normal” (thal == 3) e talvez combinar 6 e 7 em um “thal_defect” – pois 6 e 7 indicam algum tipo de defeito no exame. Isso reduz a cardinalidade e pode focar o modelo no contraste “normal vs. anormal”. Essa é uma decisão de engenharia de atributos com conhecimento médico.
- **Tratamento de valores ausentes e outliers:** embora parte disso tenha sido feito na etapa de preparação de dados, é válido reforçar aqui. Substituir valores faltantes por uma imputação apropriada (média, mediana, ou modelos de imputação) cria uma feature “limpa” para uso. Da mesma forma, detectar outliers e tratá-los (removendo ou limitando a amplitude) pode equivaler a criar uma feature binária “é_outlier” que em alguns casos melhora o modelo ou simplesmente deixar o modelo mais robusto removendo ruídos. Na nossa engenharia, supomos que já tratamos outliers

evidentes (por exemplo, valores impossíveis de certas medições) durante EDA.

Em datasets estruturados (tabulares) como o de doença cardíaca, a engenharia de atributos frequentemente é o diferencial. Profissionais experientes gastam bastante tempo iterando nessa parte, pois um atributo bem construído pode elevar significativamente a acurácia de um modelo. Entretanto, também é importante não cair na armadilha de “over-engineering”: criar centenas de variáveis complexas pode levar ao mesmo problema de overfitting que mencionamos – o modelo ajusta-se demais às peculiaridades do conjunto de treino.

Por isso, qualquer feature criada deve ser avaliada criticamente: está realmente adicionando informação útil ou só duplicando algo que outra feature já fornece? Por exemplo, se já temos *trestbps* (pressão de repouso) e *thalach* (freq. cardíaca máx.), criar “índice pressão * freq” pode não acrescentar muito se ambas já estão no modelo individualmente, pois o algoritmo (especialmente árvores ou redes neurais) pode aprender interações sozinho até certo ponto.

Outro ponto teórico: feature engineering é um processo iterativo e dependente do modelo. Às vezes, uma feature que melhora muito um modelo linear pode ser desnecessária para um modelo de árvore. Exemplo: criar manualmente uma feature binária “*chol_high*” (1 se colesterol > 250, 0 caso contrário) pode ajudar a regressão logística a captar efeito não linear do colesterol; mas uma árvore de decisão já poderia dividir naturalmente nesse ponto de corte, então para a árvore essa feature extra é redundante. Portanto, ao pensar em engenharia de atributos, considere o algoritmo alvo. Em projetos práticos, é comum primeiro definir um conjunto rico de features possíveis (brainstorming + conhecimento de domínio), depois testar modelos diferentes e simplificar o conjunto conforme necessário.

Resumindo, engenharia de atributos é a “arte” de extrair informação útil escondida nos dados brutos e apresentá-la de forma acessível ao algoritmo. É um passo muitas vezes trabalhoso (envolvendo entendimento de dados, tentativa e erro, visualizações para insight), mas extremamente recompensador: boas features não apenas melhoram modelos, como também frequentemente revelam insights de negócio (por exemplo, descobrir que certa combinação de fatores de risco é o maior preditor de doença tem valor clínico por si só).

Comparação entre Algoritmos de Modelagem

Na aula anterior escolhemos a Regressão Logística como ponto de partida por sua simplicidade e interpretabilidade. Vimos agora que, ao tentar modelos mais complexos (árvores, florestas etc.), conseguimos melhorar o desempenho. Como decidir qual algoritmo usar? A resposta curta: teste e valide, pois cada algoritmo tem seus prós e contras e não existe um modelo universalmente superior a todos – tudo depende dos dados e do problema específico. Esse é o espírito do No Free Lunch Theorem, que a grosso modo afirma que, em média, nenhum algoritmo de classificação/regressão supera os outros em todos os cenários. Assim, comparar algoritmos é parte integrante do desenvolvimento de modelos.

Vamos recapitular brevemente as características de algumas famílias de modelos, contextualizando com nosso problema de detecção de doença cardíaca:

- **Modelos Lineares** (Regressão Logística, etc.): são rápidos, simples e fáceis de interpretar (cada coeficiente indica a direção e peso do efeito de uma feature). Funcionam bem quando há uma relação aproximadamente linear entre as features. Contudo, podem falhar em capturar relações complexas – por exemplo, o risco de doença cardíaca pode não ser apenas a soma da idade e do colesterol, mas sim o produto delas, indicando que a combinação é muito mais perigosa que cada fator isolado. No dataset cardíaco, se o risco realmente aumenta de forma não linear com a idade ou colesterol (por exemplo, muito baixo ou muito alto colesterol pode ser ruim), um modelo linear terá dificuldade a menos que façamos feature engineering capturando essa não linearidade (como binning de idade etc.). Outra limitação: modelos lineares requerem pré-processamento cuidadoso – features em escalas muito diferentes ou categóricas precisam de tratamento (one-hot, escala), senão os resultados podem ficar distorcidos. Com poucas features relevantes e bem preparadas, modelos lineares frequentemente performam de maneira competitiva e têm a vantagem de extrapolar (eles conseguem fornecer uma previsão mesmo em regiões não cobertas pelos dados, embora com suposições lineares).
- **Árvores de Decisão**: algoritmos baseados em árvore (como DecisionTree) segmentam recursivamente o espaço de atributos para separar classes.

São intuitivos (podemos desenhar a árvore e explicar decisões) e capturam muito bem interações e não linearidades – por exemplo, uma árvore pode facilmente aprender regras do tipo “se $\text{thal} = \text{'Defect'}$ e $\text{idade} > 60$ então alto risco”. Elas não precisam de normalização de dados e toleram bem features irrelevantes (a árvore simplesmente não as usará nas divisões se não ajudarem). Entretanto, árvores simples tendem a sobreajustar se deixadas crescer demais – controlar profundidade ou número mínimo de amostras por folha é necessário. Em nosso caso, uma árvore pode encontrar bons limiares para variáveis como idade por conta própria. O desempenho de uma única árvore pode não ser tão alto se o problema é complexo, mas elas formam a base para métodos de ensemble.

- **Random Forest:** é um ensemble de muitas árvores de decisão treinadas em amostras aleatórias do dataset e com subconjuntos aleatórios de features em cada divisão. Essa aleatoriedade e agregação confere ao Random Forest maior poder preditivo e estabilidade que uma árvore isolada. No hands on, vimos que o Random Forest superou a regressão logística – isso é comum em muitos conjuntos de dados tabulares, pois ele consegue capturar padrões complexos sem precisar de tanta intervenção humana. Vantagens: funciona bem por padrão, não requer muito tuning para ter um resultado razoável; lida bem com features redundantes; fornece estimativas de importância de features. Desvantagens: perde em interpretabilidade (embora possamos ver as importâncias, não temos uma simples equação ou regra única – são centenas de árvores) e pode ser mais lento, especialmente se o número de árvores for grande. No cenário de saúde, a interpretabilidade é importante, mas existem técnicas para extrair explicações locais (ex.: SHAP values). No geral, ensembles como Random Forest ou Gradient Boosting (ou sua variante popular XGBoost) são muitas vezes os top performers em competições de dados estruturados. Eles, porém, requerem cuidado em ajustar hiperparâmetros (profundidade, taxa de aprendizado, etc.), pois overfitting também pode ocorrer se o modelo for muito complexo.
- **k-Nearest Neighbors (k-NN):** este é um método baseado em instâncias – para prever um novo caso, ele procura os k casos de treino mais próximos

(em termos de distância das features) e faz uma votação. É simples e pode funcionar bem em dados onde as classes formam clusters bem definidos no espaço de features. Entretanto, o k-NN sofre com a curse of dimensionality (muitos atributos tornam a noção de distância menos significativa, exigindo normalização e talvez seleção de features) e não produz um modelo “explicitamente treinado” – ele armazena todo o dataset, o que pode ser pesado, e a predição pode ser lenta se o conjunto de treino for grande. Em nosso problema, com poucas amostras, o k-NN poderia ser viável, mas em aplicações reais com milhares, ele perde eficiência. Em geral, o k-NN pode servir de baseline, mas raramente é a escolha final preferida quando outros modelos mais robustos estão disponíveis.

- **Outros modelos:** poderíamos citar Naive Bayes (muito simples e rápido, mas assume independência entre features – raramente verdade no mundo real, onde fatores geralmente são correlacionados), SVM (eficaz em certos casos complexos e dados de baixa dimensão, mas pode ser lento em datasets grandes e menos interpretável) e Redes Neurais (que seriam “overkill” para um dataset pequeno como o que utilizamos, mas que brilham em dados massivos e não estruturados como imagens e texto). Para um problema com poucas instâncias e features, redes neurais não seriam indicadas – risco alto de overfitting e necessidade de muito mais dados para treinar adequadamente.

O ponto chave é: ao desenvolver um modelo, teste múltiplas abordagens. Use a validação cruzada ou um conjunto de validação para comparar performances. Ao fazer isso, considere também outros critérios além da métrica: tempo de treinamento/predição, memória, facilidade de implementação e explicabilidade. Por exemplo, se uma regressão logística e um Random Forest têm acurácias semelhantes, a logística pode ser preferível por ser mais simples de explicar e implementar (um médico pode preferir uma fórmula clara de risco do que um modelo que não conseguimos explicar direito). Por outro lado, se o Random Forest traz 5 pontos percentuais a mais de F1-score que podem significar salvar mais vidas, talvez valha a pena sacrificar interpretabilidade – mas neste caso, você poderia investir em técnicas de explicação de modelo (LIME, SHAP). Essas decisões se baseiam no contexto de negócio: qual é a margem de erro aceitável? Precisa-se de justificativa

para cada previsão? O modelo será executado em tempo real (onde rapidez importa) ou off-line? Tudo isso faz parte da engenharia de modelos em produção.

Em nosso caso de estudo, identificamos que modelos de árvore/ensemble trouxeram ganho significativo, justificando sua adoção desde que documentamos bem seu comportamento. A título de conclusão dessa comparação, vale lembrar que diversificar algoritmos testados muitas vezes leva à ideia de combinar modelos (votos majoritários, stacking etc.) se diferentes modelos capturam aspectos distintos dos dados. Porém, isso adiciona complexidade. Na prática, um modelo bem escolhido e tunado costuma ser suficiente, mas equipes de ciência de dados frequentemente treinam vários e usam o melhor ou até fazem ensembles para competição. O importante é sempre validar com dados não vistos para ter confiança de que o modelo escolhido generaliza.

Tuning de Hiperparâmetros

Entrar no tema de tuning de hiperparâmetros é falar de buscar a “configuração perfeita” de um modelo. Hiperparâmetros são parâmetros definidos antes do treinamento (ao contrário dos parâmetros aprendidos pelo modelo). Exemplos: a constante de regularização C em uma regressão logística, a arquitetura de uma rede neural (número de camadas, neurônios), o número de vizinhos k no k -NN, profundidade máxima de árvore, número de árvores em um Random Forest etc. Esses valores podem influenciar drasticamente a performance. Um modelo mal configurado pode subaproveitar os dados; por outro lado, configurações muito complexas podem sofrer com overfitting. Por isso, fazemos tuning para encontrar um ponto ótimo de generalização.

Os dois métodos mais difundidos de tuning automatizado são a Busca Exaustiva (Grid Search) e a Busca Aleatória (Random Search). Na busca exaustiva, definimos um grid de possíveis valores para cada hiperparâmetro e o algoritmo avalia todas combinações possíveis, usando validação cruzada no conjunto de treino para estimar o desempenho de cada combinação. Em problemas reais, essa grade pode explodir em possibilidades: imagine 4 hiperparâmetros com 5 opções cada – já são 625 combinações; se cada treino leva 1 minuto, seriam mais de 10 horas de processamento. Ainda assim, a grid search garante que você não perderá a combinação exata (desde que esteja no grid).

Já a busca aleatória não tenta todas as possibilidades; em vez disso, sorteia combinações aleatórias de um dado espaço (que pode ser contínuo ou definido por distribuições em vez de listas fixas). Pesquisas mostraram que se apenas alguns hiperparâmetros importam muito e outros têm efeito menor, a busca aleatória encontra uma boa solução mais rapidamente, pois não desperdiça tempo testando todas as variações insignificantes. Por exemplo, se em um modelo dois parâmetros quase não fazem diferença e um terceiro é crucial, a chance de o sorteio pegar o valor certo do crucial em algumas dezenas de tentativas é alta, enquanto a grid teria testado muitas combinações inúteis. No Scikit-learn, o `RandomizedSearchCV` permite especificar quantas amostras aleatórias você quer avaliar.

Uma evolução dessas abordagens em anos recentes são os métodos de Halving Grid Search/Random Search, que começam testando muitas configurações rapidamente com menos recurso (por exemplo, com menos iterações ou em um subconjunto menor de dados) e iterativamente refinam a busca nas melhores candidatas usando mais recursos, “descartando” as menos promissoras – economizando tempo. Essa ideia vem do Successive Halving e da técnica chamada Hyperband. Além disso, há toda uma área de Otimização Bayesiana para hiperparâmetros, em que algoritmos (como o Tree Parzen Estimator, usado em bibliotecas como Hyperopt) modelam a função de desempenho e escolhem inteligentemente as próximas combinações a testar, focando onde pode estar o ótimo em vez de chutar aleatoriamente. Ferramentas como Optuna, Scikit-Optimize, Ax e outros implementam essas técnicas, muitas vezes encontrando hiperparâmetros ideais em menos tentativas que uma busca aleatória pura.

Seja qual for o método, alguns cuidados devem ser tomados:

- **Overfitting:** é possível, sem querer, sobreajustar os hiperparâmetros ao conjunto de validação. Isso acontece quando fazemos, por exemplo, múltiplos grid searches ou um grid muito extenso – em algum momento, uma combinação pode parecer ótima só por sorte (variância do processo). Para mitigar, costuma-se separar um conjunto de dados holdout final (que não foi usado nem no treino nem na validação do tuning) para somente no fim avaliar o modelo escolhido. Em projetos críticos, podemos fazer nested CV: um loop externo separa uma parte para teste e um interno faz o tuning

nos folds restantes, garantindo avaliação mais isenta do tuning. É custoso, mas robusto.

- **Custo Computacional:** como vimos, o tuning pode ser demorado. É interessante priorizar quais hiperparâmetros ajustar. Muitas bibliotecas têm valores padrão razoáveis baseados em experiências gerais – por exemplo, `n_estimators=100` no `RandomForest` costuma ser suficiente na maioria dos casos. Você pode então fixar alguns parâmetros nos defaults e só explorar sistematicamente os que suspeita que tenham maior impacto (como `max_depth` e `min_samples_split` no caso das árvores). Verificar a documentação ou artigos sobre o algoritmo ajuda a entender a sensibilidade de cada hiperparâmetro. Por exemplo, SVMs são muito sensíveis ao `C` (regularização) e ao parâmetro de kernel (p. ex., `gamma` no kernel RBF), então faz sentido fazer uma busca fina nesses; já o `max_iter` raramente precisa ser alterado (é mais para limitar tempo).
- **Métrica de Otimização:** ao configurar a busca, defina qual métrica quer otimizar – acurácia, F1-score, AUC, etc. Às vezes vale otimizar diretamente uma métrica de interesse do negócio. Por exemplo, se estamos mais preocupados em não perder doentes (alto recall), poderíamos usar recall como scoring no `GridSearchCV`, ou até mesmo uma métrica customizada ponderando recall e precisão. No nosso caso, poderíamos ter escolhido maximizar o F1-score, por ser o equilíbrio entre precisão e recall. Isso garante que o processo de tuning encontre o modelo com melhor F1-score médio nos folds, que pode ser diferente do de melhor acurácia.
- **Hiperparâmetros vs. Features:** às vezes, é difícil separar o tuning do modelo do tuning do conjunto de features. Escolher quais variáveis usar, normalizar ou não, etc., também são “hiperparâmetros” do pipeline. Como mostrado, podemos incluir esses passos no Pipeline e fazer grid search conjunto (por exemplo, testar: com ou sem determinada feature; `K` de features selecionadas variando; usar `StandardScaler` ou não – são opções binárias/inteiras que podem entrar no grid). Contudo, isso aumenta o grid e as combinações possíveis. Uma abordagem prática é primeiro fixar um bom conjunto de features (via análise de importância e seleção manual) e depois refinar hiperparâmetros do modelo; ou vice-versa: fixar um modelo e

encontrar um subconjunto de features ótimo para ele. Em projetos avançados, pode-se integrar ambos, mas isso demanda recursos.

No desenvolvimento atual, focamos no tuning do Random Forest. Suponha que encontramos hiperparâmetros adequados e elevamos a acurácia para 86%. Isso mostra o valor do tuning: sem alterar os dados nem o algoritmo base, apenas ajustando parâmetros, ganhamos alguns pontos percentuais de performance. Em contextos empresariais, isso pode significar milhões de reais ou vidas salvas, dependendo da aplicação, então vale o esforço! Porém, também é importante notar quando parar: hiperparâmetros têm retornos decrescentes. Pode haver apenas melhoria marginal ao gastar horas refinando de 86.0% para 86.5%, por exemplo. Avalie se vale a pena em vista do objetivo. Às vezes, é preferível adotar uma solução “suficientemente boa” que já cumpra os requisitos de negócio do que perseguir incansavelmente o último 1% (o custo computacional ou de tempo pode não compensar).

Pipelines Reproduzíveis e Validação

Durante o hands on, enfatizamos o uso de pipelines e divisão adequada de dados. Vamos nos aprofundar nesses conceitos, pois eles são a espinha dorsal de um projeto de aprendizado de máquina confiável. Um pipeline de Machine Learning é essencialmente uma sequência de passos de processamento de dados e treinamento, organizados de forma automatizada e repetível. A ideia é que, dado um novo conjunto de dados (por exemplo, novas amostras de pacientes entrando no sistema), possamos aplicar exatamente as mesmas transformações que fizemos no conjunto de treino original e em seguida usar o modelo treinado para obter previsões, tudo isso sem intervenção manual e sem erros.

Porque isso importa? Pense no fluxo completo: input raw -> pré-processamento -> features -> modelo -> output. Se cada vez que formos executar tivermos que lembrar de aplicar manualmente certas etapas, abrimos margem para falhas. Pipelines garantem disciplina e sistematização do processo. Por exemplo, no Scikit-learn, quando construímos `Pipeline([('scale', StandardScaler()), ('clf', RandomForestClassifier())])` e chamamos `fit`, internamente ele vai chamar `fit_transform` do `StandardScaler` no X de treino, depois `fit` do classificador no resultado.

Ao chamar `predict` no pipeline, ele automaticamente aplica `transform` no `StandardScaler` com os parâmetros guardados do treino e depois `predict` no classificador. Isso evita deslizos como esquecer de escalar o teste ou aplicar acidentalmente transformações usando informações do teste. Além disso, como mencionado, o pipeline se integra com ferramentas de avaliação (`cross_val_score`, `GridSearchCV`) garantindo que em cada fold de CV, por exemplo, o scaler seja ajustado somente com os dados do fold de treino e aplicado ao de validação – prevenindo vazamento de informação entre folds.

Essa abordagem padronizada de pipeline é tão importante que muitas plataformas de ML em produção exigem que você registre um pipeline completo, não apenas o modelo. Por exemplo, o MLflow (que usamos para tracking) possui a ideia de MLflow Model, onde você salva não só o modelo mas também a lógica de pré-processamento necessária, assegurando que ao servir o modelo em produção, os dados passem pelas mesmas etapas.

Em termos de reprodutibilidade, além dos pipelines, vale reforçar outras práticas: fixar sementes aleatórias/random state (como fizemos), registrar versões de dados e código e usar controle de versão (Git) para o projeto de modelagem. Empresas maduras em ciência de dados frequentemente seguem a regra de “dados entram, modelo sai, e todo o processo é versionado e automatizado”. Isso significa que alguém no futuro, ou você mesmo daqui a 6 meses, pode pegar o repositório do projeto, apontar para o mesmo dataset original (ou dataset atualizado) e recriar os resultados ou treinar um novo modelo idêntico.

Reprodutibilidade não é só comodidade – é requisito em ambientes regulados (financeiro, saúde) e para colaboração em times (você não quer depender da memória do cientista X para saber como ele preparou os dados). Por exemplo, no nosso projeto poderíamos versionar que usamos o Heart Disease UCI v1 (um hash ou data do dataset) e um commit específico do código. Se um novo dado de pacientes chega, rodamos novamente o pipeline (ou apenas aplicamos o pipeline salvo) e confiamos que as previsões são consistentes.

A validação correta dos modelos também anda junto com pipelines bem feitos. Dividir dados de forma estratificada, usando validação cruzada, separando um conjunto de teste final – esses procedimentos garantem que tenhamos uma estimativa realista de como o modelo performa fora da amostra. Vimos como um erro de

validação (por exemplo, selecionar features usando todo o dataset antes de dividir) pode introduzir vazamento e inflar indevidamente a avaliação. Já insistimos nisso, mas vale repetir: sempre pense “esta informação estaria disponível quando eu for usar o modelo de verdade?”.

Se a resposta for não, essa informação não deve ser usada no treinamento/validação. Um caso clássico: imagine que além dos atributos clínicos, tivéssemos a informação se o paciente morreu nos próximos 5 anos. Isso certamente correlaciona-se com ter doença cardíaca – mas não estaria disponível no momento do diagnóstico! Seria incorreto usá-la como feature, embora estatisticamente pudesse melhorar a acurácia. Parece óbvio neste exemplo, mas há casos sutis em que informação de futuro ou de fora do escopo vaza sem percebermos (timestamps, IDs que codificam algo etc.). A boa validação e o pipeline ajudam a evitar isso.

Por fim, um pipeline bem definido facilita a implantação do modelo. Pense: temos um único objeto que sabe “como pegar os dados de um paciente e retornar a previsão de risco”. Para integrar isso em um aplicativo, é muito mais simples do que ter que reproduzir cada etapa manualmente no código de produção. Muitas empresas utilizam orquestradores de pipelines em que você “desenha” as etapas (obtenção de dados -> pré-processamento -> treinamento -> validação -> deploy). No nosso contexto de aula, não chegamos ao deploy, mas deixamos tudo pronto: com dataset e pipeline, o passo seguinte natural seria empacotar isso em uma API ou script que alimente novos dados e devolva resultados (que veremos nas próximas aulas).

Em resumo, pipeline e validação são os pilares da engenharia de modelos responsável. Investir tempo para configurar isso corretamente compensa enormemente a confiabilidade do seu projeto.

MERCADO, CASES E TENDÊNCIAS

O rápido avanço das técnicas de desenvolvimento de modelos traz novidades constantes ao mercado. Uma tendência clara é a convergência entre Data Science e Engenharia de Software, refletida no crescimento do MLOps. Empresas têm percebido que não basta construir modelos com alta acurácia – é preciso que eles tenham processos de deployment, monitoramento e que mantenham a qualidade ao longo do tempo. Ferramentas e plataformas estão surgindo para atender a essa demanda. Por exemplo, o MLflow, que usamos de forma básica, vem sendo amplamente adotado em empresas para rastrear experimentos e gerenciar modelos em todas as fases.

Trata-se de um framework que ajuda times a estruturarem pipelines de ML de forma padronizada, modular e escalável, incorporando princípios de engenharia de software como modularidade, testabilidade e reprodutibilidade no processo de modelagem. Isso torna a transição do ambiente de experimentação para produção muito mais suave. Plataformas de nuvem como AWS, GCP e Azure também oferecem serviços integrados para pipeline – de preparação de dados até deployment – sinalizando que o futuro do desenvolvimento de modelos é cada vez mais automatizado e orientado a pipelines.

Outra grande tendência é o AutoML (Automated Machine Learning). Ferramentas de AutoML buscam automatizar etapas como seleção de features, escolha de algoritmos e tuning de hiperparâmetros, aliviando o trabalho manual do cientista de dados. No contexto de saúde, isso pode acelerar descobertas e auxiliar equipes reduzidas a obter modelos de qualidade. O AutoML não apenas poupa tempo, mas pode revelar soluções inovadoras que um humano talvez não considerasse inicialmente. Grandes players também têm soluções AutoML proprietárias (Google AutoML, H2O.ai, DataRobot etc.), muitas vezes integradas a interfaces amigáveis onde, por exemplo, um médico com conhecimento básico de dados poderia carregar um dataset e obter um modelo treinado com validação.

No mercado de tecnologia, vemos empresas que lidam com grandes volumes de dados apostando em modelos ensemble e pipelines sofisticados. Um caso divulgado é o da Uber, que desenvolveu a plataforma Michelangelo para gerenciar todo o ciclo de vida de modelos – permitindo aos engenheiros desde a extração de

features em tempo real até o treinamento e deployment de modelos ao toque de um botão. O Airbnb investiu em frameworks para experimentação rápida e verificou bons ganhos ao combinar rigor na análise exploratória com protótipos de modelo iterativos. Essas histórias ressaltam que a cultura de experimentação contínua aliada a pipelines bem construídos é uma vantagem competitiva: quem consegue iterar mais rapidamente em modelos (sem perder rastreabilidade) lança melhorias de produto mais cedo no mercado.

Especificamente na área de saúde, há um movimento forte em direção à implementação segura e transparente de modelos. Regulamentações emergentes (como as da FDA nos EUA para softwares médicos com AI) exigem que o processo de desenvolvimento de modelos seja auditável e que os modelos possam explicar suas previsões. Isso significa que aquelas boas práticas que discutimos – versionamento, documentação de features, monitoramento de performance – não são apenas luxo, mas necessidades para aprovação regulatória.

Por exemplo, um hospital que queira usar um modelo preditivo de ataque cardíaco precisará mostrar aos reguladores como o modelo foi desenvolvido, em que dados treinou, que vieses podem existir e ter um procedimento de revalidação periódica. Portanto, MLOps em saúde está crescendo: revisões sistemáticas mostram que muitos estudos estão focando em como trazer as operações de ML para o ambiente clínico de forma confiável. Empresas especializadas oferecem plataformas que mantêm modelos de ML em conformidade, monitorando se a performance cai (por exemplo, se a população de pacientes mudar com o tempo, o modelo pode precisar ser recalibrado – conceito de data drift).

Em suma, o mercado caminha para industrializar o processo de Machine Learning. Para o(a) profissional, isso significa que além de saber teoricamente sobre algoritmos, espera-se capacidade de codar com qualidade, montar pipelines, usar ferramentas de tracking e deployment. A fronteira entre “cientista de dados” e “engenheiro de ML” está cada vez mais tênue – profissionais full-stack de ML são muito demandados. Nosso exercício nesta aula refletiu esse cenário: fizemos não apenas escolhas estatísticas, mas estruturamos o trabalho de forma a ser repetível e integrável.

Essa é a mentalidade requisitada hoje. E como tendência futura, com AutoML e outras automações, o indivíduo cientista/engenheiro de ML vai se concentrar em

definir problemas, garantir qualidade dos dados e interpretar resultados, enquanto partes maçantes da otimização serão delegadas a plataformas. Ainda assim, entender os fundamentos – como fizemos explorando feature engineering, tuning e comparações – continua essencial para orientar essas ferramentas e validar seus outputs. Afinal, no fim do dia, a responsabilidade por um modelo de IA bem-sucedido é humana e envolve tanto ciência quanto engenharia.

EMAP

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprofundamos o ciclo de vida de um modelo de Machine Learning após a fase inicial de MVP. Vimos primeiramente a importância de aprimorar as variáveis de entrada e aprendemos a realizar a seleção de features, reduzindo a dimensionalidade do problema ao remover atributos pouco relevantes ou redundantes. Observamos que uma seleção correta deve ser feita utilizando apenas dados de treino, seguindo as boas práticas para evitar vazamento de informação.

Em seguida, exploramos a comparação entre diferentes algoritmos de modelagem. Também discutimos que cada família de modelos tem vantagens e limitações e que não há um modelo “coringa” para todas as situações; por isso, a experimentação e validação comparativa são essenciais no desenvolvimento. Esse processo nos levou ao tópico de tuning de hiperparâmetros, em que utilizamos busca em grade (grid search) para encontrar configurações ótimas do modelo escolhido.

Durante todo o desenvolvimento, enfatizamos a construção de pipelines reproduzíveis. Estruturamos as etapas de pré-processamento, seleção de features e modelagem em um único fluxo integrado (usando o recurso de Pipeline do Scikit-learn), garantindo consistência nas transformações aplicadas e evitando erros de procedimento. Também incorporamos boas práticas de codificação e organização, seguindo uma mentalidade de engenharia de software.

Em síntese, nesta aula você viu como evoluir de um protótipo básico para um modelo robusto e pronto para produção. Aprendeu que melhorar um modelo envolve tanto arte quanto ciência: arte ao criar features inteligentes e selecionar algoritmos adequados com base no conhecimento do problema, e ciência ao testar hipóteses de forma rigorosa, medir resultados em dados de validação e aplicar procedimentos estatísticos sólidos (como cross-validation e grid search).

Você também experimentou na prática o efeito de técnicas avançadas – do aumento de acurácia com novas features e modelos mais complexos até a conveniência de pipelines automatizados para evitar armadilhas de validação. Por fim, conectamos esses conceitos ao panorama do mercado, entendendo que as habilidades de engenharia de modelos são altamente valorizadas: combinar conhecimento de algoritmos com capacidade de implementar soluções de forma limpa

e escalável. Com isso, concluímos a etapa de Desenvolvimento e Engenharia de Modelos, prontos para nas próximas aulas abordar fases como implantação, monitoramento e governança, completando assim a jornada do aprendizado de máquina do início ao fim.

EMAP

REFERÊNCIAS

LABEL YOUR DATA TEAM. **Pattern Recognition and Machine Learning: 2025 Industry Applications.** 2025. Disponível em: <https://labeledyourdata.com/articles/pattern-recognition-in-machine-learning>. Acesso em: 28 jan. 2026.

LE, T. et al. **The tree-based pipeline optimization tool: Tackling biomedical research problems with genetic programming and automated machine learning.** 2025. Disponível em: <https://www.sciencedirect.com/>. Acesso em: 28 jan. 2026.

MLFLOW. **MLflow.** 2025. Disponível em: <https://mlflow.org/>. Acesso em: 28 jan. 2026.

PATI, S. K. **Best Practices in Data Science — Part 1 (Organizing and Coding).** 2025. Disponível em: <https://medium.com/thedeephub/best-practices-for-organizing-and-coding-data-science-projects-part-1-72539e14a7a0>. Acesso em: 28 jan. 2026.

SCIKIT-LEARN. **Common pitfalls and recommended practices.** 2023. Disponível em: https://scikit-learn.org/stable/common_pitfalls.html. Acesso em: 28 jan. 2026.

SCIKIT-LEARN. **Feature selection.** 2023. Disponível em: https://scikit-learn.org/stable/modules/feature_selection.html. Acesso em: 28 jan. 2026.

SCIKIT-LEARN. **Tuning the hyper-parameters of an estimator.** 2023. Disponível em: https://scikit-learn.org/stable/modules/grid_search.html. Acesso em: 28 jan. 2026.

SONY, R. K. **Heart Disease Data – Kaggle.** 2023. Disponível em: <https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data/data>. Acesso em: 28 jan. 2026.

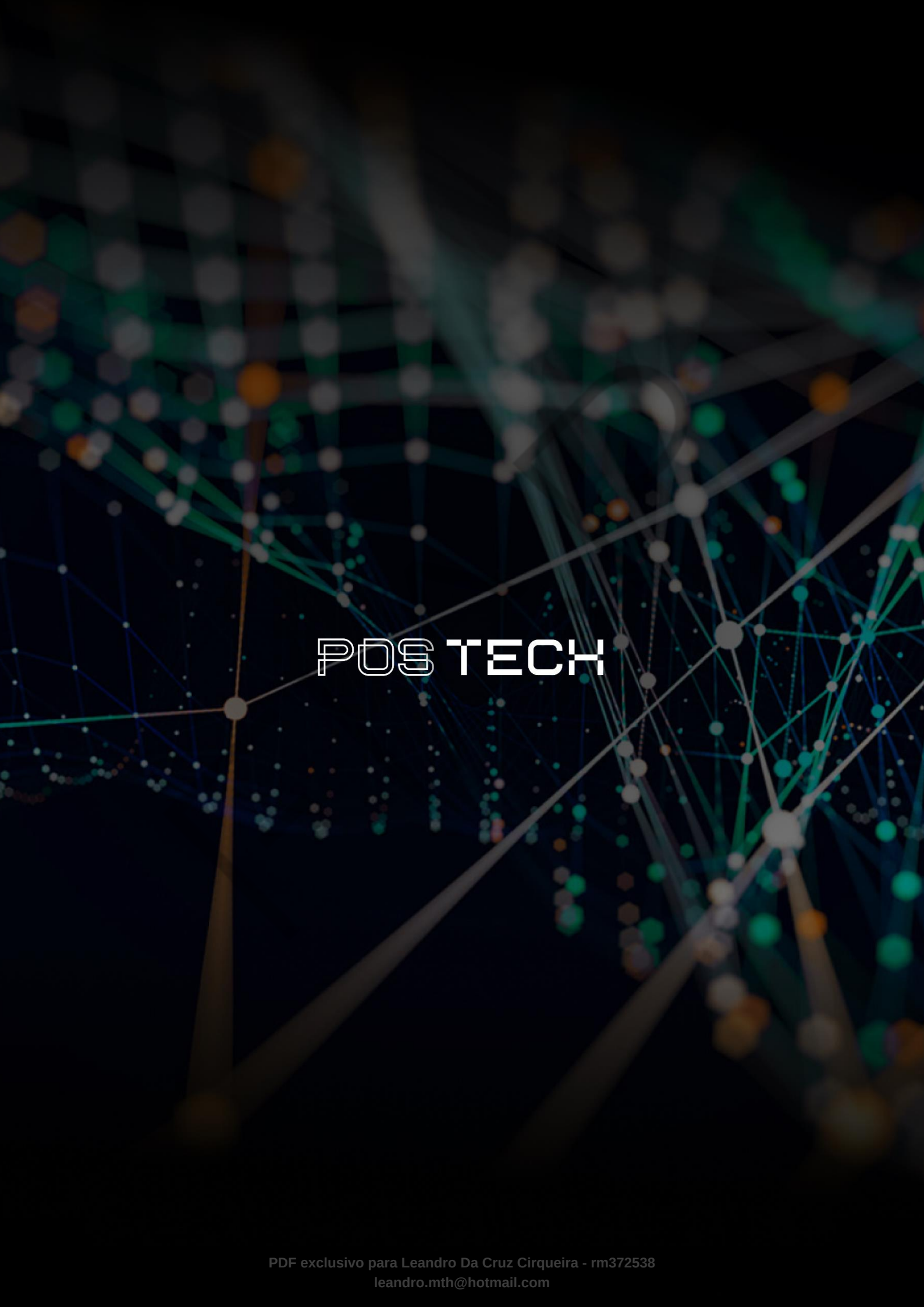
UCI MACHINE LEARNING REPOSITORY. **Heart Disease.** 1988. Disponível em: <https://archive.ics.uci.edu/dataset/45/heart+disease>. Acesso em: 28 jan. 2026.

WHITFIELD, B. **Feature Engineering Explained.** 2025. Disponível em: <https://builtin.com/articles/feature-engineering>. Acesso em: 28 jan. 2026.

PALAVRAS-CHAVE

Feature Engineering. Tuning de Hiperparâmetros. Pipelines de ML.

EMEND



POSTECH